



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
شبکه‌های عصبی و یادگیری عمیق
تمرین سوم

نام و نام خانوادگی	امیرحسین راحتی
شماره دانشجویی	۸۱۰۱۰۴۳۰۳
تاریخ ارسال گزارش	۲ خرداد ۱۴۰۵

فهرست

سؤال اول 3

3..... بخش اول: سوالات تئوری

3..... زیربخش اول: مدل های دو مرحله ای

3..... زیربخش دوم: مقایسه FAST RCNN و FASTER RCNN

4..... زیربخش سوم: معماری FASTER RCNN

4..... زیربخش چهارم: معماری Sparse RCNN

5..... بخش دوم: سوالات پیاده سازی

5..... زیربخش اول: آماده سازی داده ها

6..... زیربخش دوم: تقسیم بندی داده ها و ساخت dataloader

7..... زیربخش سوم: پیاده سازی معماری Sparse RCNN

8..... زیربخش چهارم: تعریف تابع هزینه و تطبیق دو طرفه

9..... زیربخش پنجم: تعیین هایپرپارامتر ها و آموزش

11..... زیربخش ششم: ارزیابی نهایی و تحلیل

14مراجع

بخش اول: سوالات تئوری

زیربخش اول: مدل های دو مرحله ای

الف) R-CNN و FASTER RCNN و FAST RCNN و SPATIAL PYRAMID POOLING

ب) در روش های دو مرحله ای، ابتدا مرحله اول یک مجموعه از Region Proposals را بدون توجه به کلاس اشیاء تولید می کند. سپس مرحله دوم، هر ناحیه را جداگانه طبقه بندی کرده و موقعیت آن را دقیق تر می کند اما در روش های یک مرحله ای، شبکه مستقیماً در یک گذر از تصویر، هم bounding boxes و هم کلاس اشیاء را همزمان پیش بینی می کند و مرحله مجزایی برای تولید proposal وجود ندارد.

ج) مزیت: دقت بالاتر در تشخیص اشیاء به خصوص برای اشیاء کوچک یا آنهایی که همپوشانی دارند. به دلیل وجود مرحله دوم که روی هر proposal به صورت مجزا عمل می کند، شبکه می تواند ویژگی های دقیق تری از هر ناحیه استخراج کند.

عیب: سرعت کمتر و هزینه محاسباتی بالاتر نسبت به روش های یک مرحله ای. چون شبکه باید ابتدا proposal های زیادی تولید کند و سپس هر کدام را دوباره پردازش کند، تعداد کل عملیات بیشتر میشود.

زیربخش دوم: مقایسه FAST RCNN و FASTER RCNN

الف) معماری FAST RCNN: ابتدا کل تصویر به شبکه وارد می شود. سپس تصویر از چند لایه convolutional عبور می کند تا یک feature map تولید شود. سپس از یک روش Rule based می آید و region proposal دریافت می کند. در مرحله ROI POOLING هر proposal روی feature map بریده شده و به اندازه ثابت تبدیل می شود. در نهایت این ویژگی ها به دو مسیر می روند: یکی برای پیش بینی کلاس و یکی برای اصلاح باکس ها معماری Faster R-CNN: تصویر از لایه های convolutional عبور می کند تا feature map تولید شود. سپس در RPN روی همان feature map یک سری کاندیدا ایجاد میکند. در مرحله ROI pooling کاندیدا های تولید شده به اندازه ثابت تبدیل می شوند. بقیه مراحل مشابه fast RCNN است.

ب) مزیت اول: استفاده از روش های مبتنی بر learning به جای روش های Rule based برای استخراج کاندیداها: این کار با Region Proposal Network که مستقیماً روی feature map اجرا می شود انجام میشود. این موضوع باعث میشود مدل بتواند بر اساس داده ها فیچر مناسب استخراج کند.

مزیت دوم: یادگیری یکپارچه کل شبکه: در Fast R-CNN، بخش تولید proposal از شبکه جدا بود و قابل آموزش با backpropagation نبود. در Faster R-CNN با اضافه کردن RPN، کل شبکه شامل بخش استخراج ویژگی، RPN و head با یک loss مشترک آموزش داده می شود.

زیربخش سوم: معماری FASTER RCNN

الف) anchor ها یا کس های مرجع از پیش تعریف شده ای هستند با اندازه ها و نسبت های ابعاد مختلف. نقش آنها این است که به Region Proposal Network یاد بدهند اشیاء با اندازه و شکل های گوناگون را تشخیص دهد. به این صورت که روی هر نقطه از feature map ، چندین anchor با ابعاد متفاوت قرار میگیرد. شبکه برای هر anchor پیش بینی میکند که آیا داخل آن یک شیء وجود دارد یا نه و همچنین مختصات آن را نسبت به anchor اصلاح میکند.

ب) این روش برای حذف جعبه های اضافی و تکراری استفاده میشود. زمانی که یک شیء در تصویر وجود دارد، معمولاً چندین bounding box دور آن تشکیل میشود. این روش به این صورت عمل میکند که جعبه ها را بر اساس امتیاز confidence مرتب میکند، جعبه با بالاترین امتیاز را انتخاب کرده و سپس تمام جعبه های دیگر که همپوشانی زیادی با آن دارند مثلاً Intersection over Union یا IoU بیشتر از یک آستانه مثل ۰/۵ را حذف میکند. این کار را تکرار میکند تا همه جعبه های باقیمانده پردازش شوند. در نهایت برای هر شیء فقط یک جعبه با بالاترین اطمینان باقی میماند.

ج) RPN برای هر نقطه از feature map چندین anchor تولید میکند. بسیاری از این anchor ها روی یک آبجکت واحد همپوشانی شدیدی دارند. اگر NMS اعمال نشود، برای یک شیء واحد چند باکس خروجی گرفته میشود که بیشتر آنها اضافی هستند و باعث ایجاد نتایج تکراری و غیرقابل استفاده میشوند.

د) چون روی تمام نقاط feature map با فواصل منظم و با چندین اندازه و نسبت ابعاد در هر نقطه قرار میگیرند. یعنی عملاً کل فضای تصویر را با چگالی بالا پوشش میدهند به طوری که بین anchor های مجاور overlap زیادی وجود دارد. مشکل dense بودن این است که تعداد بسیار زیادی anchor تولید میشود که بیشتر آنها پس زمینه هستند. این مسئله باعث عدم تعادل شدید بین نمونه های مثبت و منفی میشود و شبکه را در یادگیری دچار مشکل میکند. همچنین هزینه محاسباتی بسیار بالایی دارد چون باید برای هر anchor محاسبات انجام شود. علاوه بر این برای کاهش تعداد proposal ها ناچار به استفاده از NMS هستیم که یک مرحله پس پردازشی دستکاری شده و غیرقابل یادگیری است و ممکن است در برخی موارد proposal های خوب را هم حذف کند.

زیربخش چهارم: معماری Sparse RCNN

الف) در روش های dense proposal ، ha توسط RPN و با استفاده از تعداد زیادی anchor متراکم تولید می شوند. اما در Sparse R-CNN خبری از RPN و anchor های dense نیست. در عوض، تعداد ثابت و کمی از باکس های قابل یادگیری مستقیماً به عنوان پارامتر شبکه تعریف می شوند و در طول آموزش به سمت اشیاء حرکت می کنند.

ب) learnable proposal box یک بردار چهار بعدی شامل مختصات نرمالیزه شده جعبه است و نقش تخمین اولیه از محل قرارگیری شیء را دارد.

learnable proposal feature یک بردار با ابعاد بالاست که خصوصیات ظاهری و شکل شیء را در خود جای می دهد.

ج) به تدریج موقعیت جعبه ها و ویژگی های آنها را در چند مرحله بهبود می بخشد. در هر مرحله، ابتدا ROI Align روی feature map با مختصات جعبه های فعلی انجام می شود و سپس ویژگی های استخراج شده با

proposal feature ترکیب می‌گردند. سپس دسته‌بندی و جعبه اصلاح شده جدیدی برای مرحله بعد تولید می‌شود.

د) در Faster R-CNN به دلیل وجود چندین anchor متراکم، برای یک شیء جعبه‌های تکراری زیادی تولید می‌شود و ناچار به استفاده از NMS هستیم. اما در Sparse R-CNN تعداد proposal ها بسیار کم و محدود است. همچنین bipartite matching در حین آموزش باعث می‌شود هر proposal مسئول یک شیء خاص شود و جعبه‌های تکراری به طور طبیعی حذف گردند.

بخش دوم: سوالات پیاده سازی

زیربخش اول: آماده سازی داده ها

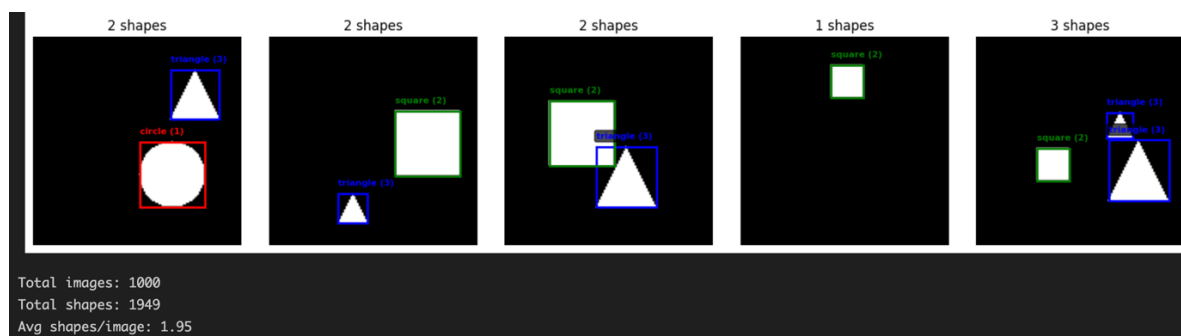
در این قسمت ابتدا توابعی برای رسم اشکال هندسی روی یک مختصات دو بعدی نوشته شد. سپس تصاویری با پس زمینه مشکی ایجاد شد که داخل آن ها از این اشکال هندسی با تعداد و ابعاد و لوکیشن رندوم ایجاد شد.

```
def draw_circle(img, x, y, size):
    cx, cy = x + size//2, y + size//2
    r = size//2
    for i in range(y, min(y+size, IMG_SIZE)):
        for j in range(x, min(x+size, IMG_SIZE)):
            if (i-cy)**2 + (j-cx)**2 <= r**2:
                img[i, j] = 255
    return img

def draw_square(img, x, y, size):
    img[y:min(y+size, IMG_SIZE), x:min(x+size, IMG_SIZE)] = 255
    return img
```

شکل 1- توابع رسم دایره و مربع

شکل زیر چند نمونه از این داده ها نمایش داده شده است. همچنین باکس اصلی و مورد انتظار این تصاویر همانطور که در تصویر مشخص است رسم شده است به عنوان bounding box در کنار بقیه داده های مورد نیاز برای آموزش مدل در دیتاست ذخیره شده است.



شکل 2- چند نمونه از داده های ایجاد شده

همچنین تعداد کل اشکال هندسی و میانگین آن ها در هر شکل نمایش داده شده است. برای اینکه اشکال حین ایجاد بیشتر از هم روی همدیگر قرار نگیرند، یک تابع وظیفه بررسی این موضوع را حین ایجاد دارد تا اگر اشتراک با شکل دیگری بیشتر از threshold بود دوباره به صورت رندوم یک شکل جدید ایجاد کند.

```
def check_overlap(b1, b2):
    x1, y1, w1, h1 = b1
    x2, y2, w2, h2 = b2
    xi1, yi1 = max(x1, x2), max(y1, y2)
    xi2, yi2 = min(x1+w1, x2+w2), min(y1+h1, y2+h2)
    inter = max(0, xi2-xi1) * max(0, yi2-yi1)
    area1, area2 = w1*h1, w2*h2
    iou = inter / (area1 + area2 - inter) if (area1 + area2 - inter) > 0 else 0
    return iou > 0.1
```

شکل 3- بررسی همپوشانی دو شکل هندسی

زیربخش دوم: تقسیم بندی داده ها و ساخت dataloader

داده ها به صورت زیر به دو دسته train و test تقسیم شده اند.

```
DATA_SPLIT_RATIO = 0.9
NUM_TRAIN_DATA = int(DATA_SPLIT_RATIO * NUM_IMAGES)
X_train, X_test, y_train, y_test = list(), list(), list(), list()

for i in range(NUM_TRAIN_DATA):
    X_train.append(dataset[i]["image"])
    y_train.append(dataset[i]["shapes"])

for i in range(NUM_TRAIN_DATA, NUM_IMAGES):
    X_test.append(dataset[i]["image"])
    y_test.append(dataset[i]["shapes"])
```

شکل 4- تقسیم داده ها به train و test

برای ساخت dataloader ابتدا کلاس ImgDataset را ایجاد میکنیم که از کلاس dataset در dataloader ارث بری میکند تا بتوان از آن برای ایجاد dataloader استفاده کرد. در متد __getitem__ داده ها و لیبل به تنسور تبدیل میشوند.

```
class ImgDataset(Dataset):
    def __init__(self, features_list: list[dict], labels_list: list[dict]):
        self.features = features_list
        self.labels = labels_list

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        x = torch.tensor(self.features[idx], dtype = torch.float32)
        y = []
        for shape in self.labels[idx]:
            bbox = shape['bbox']
            label = shape['label']
            y.append({
                'boxes': torch.tensor(bbox, dtype=torch.float32),
                'labels': torch.tensor([label], dtype=torch.int64)
            })
        return x, y
```

شکل 5 - ایجاد ImgDataset

```
train_dataset = ImgDataset(X_train, y_train)
test_dataset = ImgDataset(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size= 16, shuffle= True, collate_fn=collate_fn)
test_loader = DataLoader(test_dataset, batch_size= 16, shuffle= True, collate_fn=collate_fn)
```

شکل 6 - ایجاد Dataloader

زیربخش سوم: پیاده سازی معماری Sparse RCNN

برای پیاده سازی backbone از ۳ لایه کانولوشن دو بعدی استفاده کردیم. ابتدا اندازه in_channel یک است. در هر لایه به ترتیب اندازه چنل به ۳۲ و ۶۴ و ۱۲۸ میرسد. همچنین ابعاد تصویر به خاطر وجود لایه های maxpool ۲در۲ که ۳ نمونه از آن وجود دارد در نهایت به ۱۶در۱۶ خواهد رسید.

```
self.backbone = nn.Sequential(  
    nn.Conv2d(in_channel, 32, kernel_size=3, padding=1),  
    nn.BatchNorm2d(32),  
    nn.ReLU(True),  
    nn.MaxPool2d(2, 2),  
    nn.Conv2d(32, 64, kernel_size=3, padding=1),  
    nn.BatchNorm2d(64),  
    nn.ReLU(True),  
    nn.MaxPool2d(2, 2),  
    nn.Conv2d(64, out_channel, kernel_size=3, padding=1),  
    nn.BatchNorm2d(out_channel),  
    nn.ReLU(True),  
    nn.MaxPool2d(2, 2)  
)
```

شکل ۷- پیاده سازی قسمت Backbone شبکه

برای پارامترهای proposal box یک لیست ۴ بعدی به تعداد پروپوزال های مورد نیاز ایجاد شد که هر عدد به ترتیب نشان دهنده x, y, h, w است. چون اعداد ایجاد شده در بازه ۰ تا ۱ هستند، برای تخمین بهتر و بهتر train شدن شبکه همه اعداد در ۶۴ (نصف ابعاد عکس ها) ضرب شده است. ابعاد فیچر پروپوزال هم به همان تعداد و به اندازه تعداد فیچر های مورد نیاز ایجاد شدند.

```
self.proposal_boxes = nn.Parameter(torch.rand(num_proposals, 4) * (IMG_SIZE / 2))  
self.proposal_features = nn.Parameter(torch.randn(num_proposals, num_features))
```

شکل ۸ - ایجاد پارامترهای Proposal box و Proposal feature

در بخش InteractiveHead، در متد forward، ابتدا roi_features که به شکل چندبعدی هستند به بردارهای تخت تبدیل می شوند. سپس proposal_features از یک لایه خطی به نام weight_generator عبور می کند تا وزن های تعاملی تولید شود و این وزن ها پس از اعمال سیگموئید به صورت عنصر به عنصر با ویژگی های تخت شده ضرب می گردند. در نهایت خروجی حاصل به دو شاخه جداگانه شامل classifier برای پیش بینی کلاس اشیاء به همراه پس زمینه و box_regressor برای پیش بینی اصلاحات مختصات جعبه ها داده می شود.

```

class DynamicInteractiveHead(nn.Module):
    def __init__(self, in_channels: int, hidden_dim: int, num_classes: int, roi_output_size: tuple):
        super().__init__()

        self.roi_output_size = roi_output_size
        self.in_channels = in_channels
        self.hidden_dim = hidden_dim

        flattened_size = in_channels * roi_output_size[0] * roi_output_size[1]
        self.weight_generator = nn.Linear(hidden_dim, flattened_size)

        self.classifier = nn.Sequential(
            nn.Linear(flattened_size, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, num_classes + 1)
        )

        self.box_regressor = nn.Sequential(
            nn.Linear(flattened_size, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 4)
        )

```

شکل - 9 ساختار بخش *Dynamic head*

زیربخش چهارم: تعریف تابع هزینه و تطبیق دو طرفه

کلاس HungarianMatcher مسئول تطبیق دوطرفه بین پیش‌بینی‌های مدل و باکس صحیح است. در متد forward، برای هر تصویر در batch، ماتریس هزینه ترکیبی از دو بخش loss طبقه بندی و loss جای صحیح جعبه ساخته می‌شود. هزینه طبقه بندی با احتمال پیش‌بینی شده برای کلاس هدف محاسبه می‌شود و هزینه باکس با فاصله منتهن بین جعبه‌های پیش‌بینی شده و باکس صحیح بدست می‌آید. سپس الگوریتم مجارستانی یا linear_sum_assignment بهینه‌ترین جفت‌سازی بین پیش‌بینی‌ها و اهداف را پیدا می‌کند و ایندکس‌های متناظر را باز می‌گرداند

```

class HungarianMatcher(nn.Module):
    def __init__(self, class_cost=1, bbox_cost=5):
        super().__init__()
        self.class_cost = class_cost
        self.bbox_cost = bbox_cost

    @torch.no_grad()
    def forward(self, outputs, targets):

        batch_size, num_queries = outputs['pred_logits'].shape[:2]
        indices = []

        for batch_idx in range(batch_size):
            predicted_probabilities = outputs['pred_logits'][batch_idx].softmax(-1)
            predicted_boxes = outputs['pred_boxes'][batch_idx]
            if len(targets[batch_idx]) == 0:
                indices.append((torch.tensor([]), torch.tensor([])))
                continue
            target_class_ids = torch.cat([target['labels'] for target in targets[batch_idx]])
            target_boxes = torch.stack([target['boxes'] for target in targets[batch_idx]])
            classification_cost = -predicted_probabilities[:, target_class_ids]
            box_cost = torch.cdist(predicted_boxes, target_boxes, p=1)
            total_cost = self.class_cost * classification_cost + self.bbox_cost * box_cost
            assigned_predictions, assigned_targets = linear_sum_assignment(total_cost.cpu())

            indices.append((torch.as_tensor(assigned_predictions),
                            torch.as_tensor(assigned_targets)))

        return indices

```

شکل 10 - ماژول HungarianMatcher

کلاس تابع هزینه نهایی را برای آموزش مدل محاسبه می‌کند. در متد `forward`، ابتدا با استفاده از `HungarianMatcher` جفت‌سازی بین پیش‌بینی‌ها و اهداف انجام می‌شود. سپس بر اساس این جفت‌سازی، یک آرایه `target_classes` و `target_boxes` ساخته می‌شود که در آن پیش‌بینی‌های بدون تطبیق به عنوان پس‌زمینه در نظر گرفته می‌شوند. خطای طبقه بندی با استفاده از `classification Loss` محاسبه می‌شود که برای مقابله با عدم تعادل شدید بین نمونه‌های مثبت و پس‌زمینه طراحی شده است. خطای لوکیشن جعبه نیز با `L1 Loss` فقط روی پیش‌بینی‌هایی که به یک شیء غیر پس‌زمینه منتسب شده‌اند محاسبه می‌شود. در نهایت مجموع وزنی این دو هزینه به عنوان `loss` نهایی برگردانده می‌شود.

```
def forward(self, outputs, targets):
    indices = self.matcher(outputs, targets)

    batch_size = len(targets)
    num_queries = outputs['pred_logits'].shape[1]

    src_logits = outputs['pred_logits']
    src_boxes = outputs['pred_boxes']

    target_classes = torch.zeros((batch_size, num_queries), dtype=torch.int64, device=src_logits.device)
    target_boxes = torch.zeros((batch_size, num_queries, 4), device=src_boxes.device)

    for b, (src_idx, tgt_idx) in enumerate(indices):
        if len(tgt_idx) > 0:
            target_classes[b, src_idx] = torch.cat([targets[b][i]['labels'] for i in tgt_idx])
            target_boxes[b, src_idx] = torch.stack([targets[b][i]['boxes'] for i in tgt_idx])

    class_loss = self._focal_loss(src_logits, target_classes)

    pos_mask = target_classes != 0
    if pos_mask.sum() > 0:
        bbox_loss = F.l1_loss(src_boxes[pos_mask], target_boxes[pos_mask])
    else:
        bbox_loss = torch.tensor(0.0, device=src_boxes.device)
    total_loss = self.class_weight * class_loss + self.bbox_weight * bbox_loss
    return {'loss_total': total_loss, 'loss_class': class_loss, 'loss_bbox': bbox_loss}
```

شکل 11- کلاس `SetCriterion`

زیربخش پنجم: تعیین هایپرپارامترها و آموزش

در این قسمت از مدل یک `instance` ایجاد شده است. همچنین از بهینه ساز `Adam` با نرخ یادگیری خواسته شده استفاده شده. نمونه هایی هم برای `HungarianMatcher` و `lossCriterion` ایجاد شد که ضریب خطا برای `box` ها ۵ و برای خطای طبقه بندی 2 در نظر گرفته شد

```
model = Sparse_RCNN(
    in_channel=1,
    out_channel=128,
    num_proposals=NUM_PROPOSALS,
    num_features=HIDDEN_DIM,
    num_classes=NUM_CLASSES,
    img_size=IMG_SIZE,
    num_iterations=NUM_ITERATIONS
).to(device)

optimizer = torch.optim.Adam(model.parameters(), lr=5e-5)
matcher = HungarianMatcher(class_cost=2, bbox_cost=5)
criterion = LossCriterion(num_classes=NUM_CLASSES, matcher=matcher,
                           class_weight=2.0, bbox_weight=5.0)

num_epochs = 40
train_losses = []
```

شکل 12- ایجاد کلاس ها برای آموزش مدل

آموزش در یک حلقه با ۴۰ اپیاک انجام شد و در هر مرحله هر دو خطا بهینه سازی شده است:

```
for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0.0

    for images, targets in train_loader:
        images = images.to(device)
        class_logits, pred_boxes = model(images)
        outputs = {'pred_logits': class_logits, 'pred_boxes': pred_boxes}
        loss_dict = criterion(outputs, targets)
        loss = loss_dict['loss_total']
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()

    train_losses.append(epoch_loss)
    print(f'Epoch {epoch+1}/{num_epochs}, Loss: {epoch_loss:.4f}')
```

شکل ۱۳ - حلقه آموزش مدل

در نهایت نمودار خطای مدل و میزان خطا در هر اپیاک به صورت زیر است:

Epoch 1/40, Loss: 1537.3291	Epoch 31/40, Loss: 437.8275
Epoch 2/40, Loss: 1054.6544	Epoch 32/40, Loss: 462.7200
Epoch 3/40, Loss: 834.3740	Epoch 33/40, Loss: 461.7606
Epoch 4/40, Loss: 822.1762	Epoch 34/40, Loss: 460.3483
Epoch 5/40, Loss: 803.7207	Epoch 35/40, Loss: 446.9423
Epoch 6/40, Loss: 690.4868	Epoch 36/40, Loss: 408.9071
Epoch 7/40, Loss: 705.4929	Epoch 37/40, Loss: 379.4247
Epoch 8/40, Loss: 685.0334	Epoch 38/40, Loss: 395.9267
Epoch 9/40, Loss: 733.5130	Epoch 39/40, Loss: 455.4638
Epoch 10/40, Loss: 702.6822	Epoch 40/40, Loss: 416.5158

شکل ۱۴ - اندازه خطای آموزش



شکل ۱۵ - نمودار خطای آموزش در هر اپیاک

زیربخش ششم: ارزیابی نهایی و تحلیل

در بخش ارزیابی، معیارهایی مثل mAP و iou مورد استفاده قرار گرفتند. همچنین برخی توابع کمکی برای نمایش بهتر نتایج و تبدیل نتایج برای مشاهده استفاده شد.

```
def calculate_iou(box1, box2):
    box1_tensor = torch.tensor([box1], dtype=torch.float32)
    box2_tensor = torch.tensor([box2], dtype=torch.float32)
    iou_value = box_iou(box1_tensor, box2_tensor).item()
    return iou_value
```

شکل 16 - محاسبه iou

تابع زیر مقدار AP را برای یک کلاس مشخص محاسبه می‌کند. ابتدا پیش‌بینی‌ها بر اساس امتیاز confidence به صورت نزولی مرتب می‌شوند و سپس برای هر پیش‌بینی، بهترین تطابق با هدف‌های حقیقت زمین از نظر Intersection over Union یا IoU پیدا می‌شود. اگر بهترین IoU از آستانه تعیین شده بیشتر باشد، آن پیش‌بینی به عنوان true positive در نظر گرفته می‌شود و در غیر این صورت false positive خواهد بود. در نهایت با استفاده از جمع true positive و false positive، منحنی ساخته شده و AP با روش درونیابی ۱۱ نقطه‌ای که میانگین precision در یازده آستانه recall از صفر تا یک با گام ۰.۱ است محاسبه می‌گردد.

```
def calculate_ap(predictions, targets, iou_threshold=0.5):
    if len(predictions) == 0:
        return 0.0 if len(targets) > 0 else 1.0

    predictions = sorted(predictions, key=lambda x: x['score'], reverse=True)
    tp = np.zeros(len(predictions))
    fp = np.zeros(len(predictions))
    matched_targets = set()
    for i, pred in enumerate(predictions):
        best_iou = 0
        best_j = -1

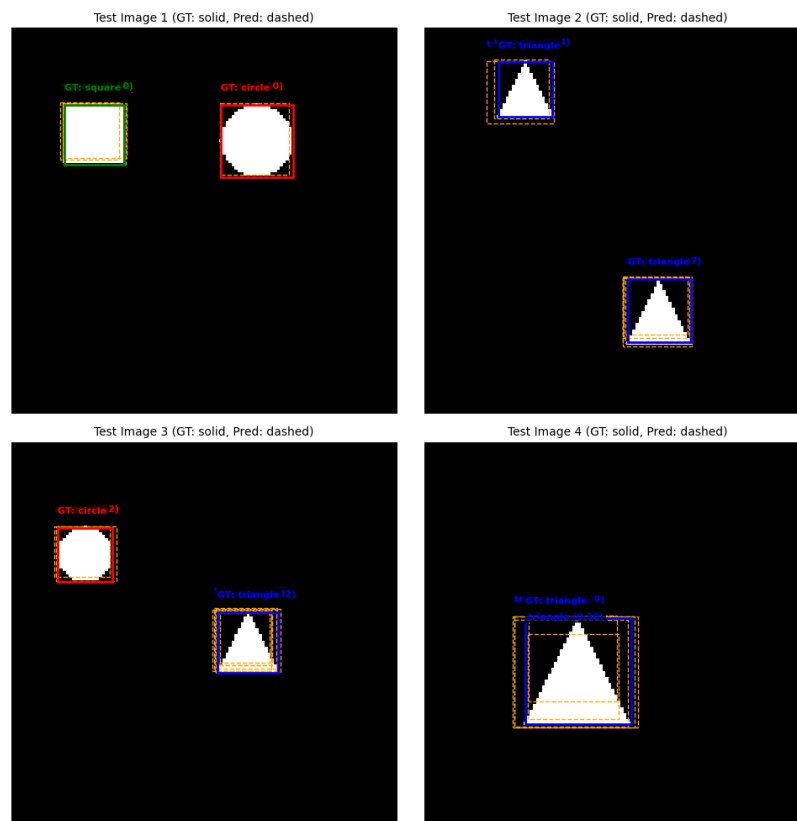
        for j, target in enumerate(targets):
            if j in matched_targets:
                continue
            iou = calculate_iou(pred['box'], target['box'])
            if iou > best_iou:
                best_iou = iou
                best_j = j

        if best_iou >= iou_threshold:
            tp[i] = 1
            matched_targets.add(best_j)
        else:
            fp[i] = 1

    tp_cumsum = np.cumsum(tp)
    fp_cumsum = np.cumsum(fp)
    recalls = tp_cumsum / len(targets) if len(targets) > 0 else np.zeros_like(tp_cumsum)
    precisions = tp_cumsum / (tp_cumsum + fp_cumsum + 1e-6)
```

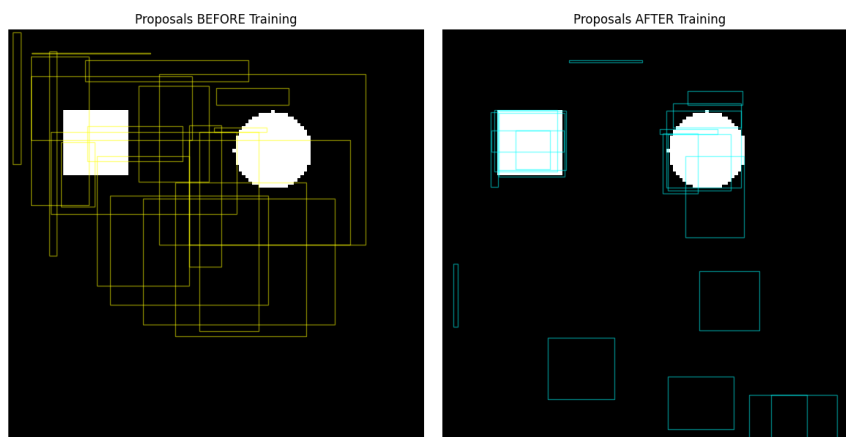
شکل 17 - محاسبه AP

در نهایت مدل روی داده‌های تست اجرا می‌شود و به عنوان نمونه می‌توان چند مورد از پیش‌بینی‌های مدل را مشاهده کرد:



شکل 18- نتایج روی چند نمونه از داده های تست

همانطور که مشاهده میشود ، خط چین های نارنجی رنگ دور هر شکل نشان دهنده بهترین پیشبینی های انجام شده روی نمونه هستند که تا حد خوبی با ground truth مطابقت دارد. همچنین در شکل زیر میتوان یک نمونه از داده های تست را مشاهده کرد که در آن همه box ها در مرحله اول پیش از آموزش و در مرحله پس از آموزش مدل مشاهده میشود



شکل 19- تغییرات محل باکس ها قبل و بعد از اجرای مدل

مشاهده میشود که باکس ها به میزان مناسبی جابجا شده اند و مدل به طور مطلوبی میتواند اشکال داخل تصاویر را تشخیص دهد و باکس مناسب برای آن را تخمین بزند. همچنین نتایج معیار mAP برای هر کلاس به صورت زیر است:

Average Precision per class:

Circle : 0.6895

Square : 0.7571

Triangle : 0.7497

mAP : 0.7321

شکل 20 - محاسبه معیار mAP برای هر کلاس

این نتایج نشان می‌دهد مدل شما روی مجموعه تست عملکرد نسبتاً خوبی داشته است. مقدار mAP برابر ۰.۷۳۲۱ به این معناست که به طور متوسط دقت تشخیص مدل در شناسایی و مکانیابی هر سه شکل هندسی قابل قبول است. از بین سه کلاس، مربع با مقدار ۰.۷۵۷۱ بهترین عملکرد و دایره با مقدار ۰/۶۸۹۵ ضعیف‌ترین عملکرد را داشته است که این تفاوت می‌تواند ناشی از چالش در تشخیص مرزهای منحنی توسط ویژگی‌های استخراج شده باشد.

